Worksheet: Optimization & Parametrization Prep

Topic: Maximal Update Parameterization (μP), RMS Norm, and Optimizer Scales
Time: ~10 Minutes

Part 1: Key Concepts Cheat Sheet

Quick refreshers based on Lectures 5, 6, and 7.

1. The Optimizer "Recipe"

Optimization algorithms can be derived by solving a dual problem involving a specific norm:

$$\text{argmin} \Delta W \langle \nabla W L(W), \Delta W \rangle \quad \text{subject to} \quad \|\Delta W\| \leq \eta$$

- **SignSGD** (Adam-like) arises from using the $\ell\infty$ norm constraint.
- **Gradient Descent** arises from using the $\ell 2$ (Frobenius) norm constraint.
- **Shampoo** arises from using the Spectral norm constraint.

2. RMS Norm (Root Mean Square)

Unlike the Frobenius norm which sums squares, the RMS norm averages them, making it independent of dimension size (in expectation):

$$\|A\|RMS = \frac{\|A\|F}{\sqrt{dindout}}$$

- **Motivation:** Standard spectral norms or Frobenius norms grow with width, making hyperparameter tuning difficult across different model sizes.

3. Maximal Update Parameterization (μP)

- **Goal:** Optimize hyperparameters on small networks and "transfer" them to large networks.
- **Mechanism:** Adjust the learning rate scaling based on layer dimensions ($din, dout$) so that the change in feature activations (Δh) is $O(1)$ regardless of width.

Part 2: Math "Warmup" Drills

These questions isolate the mathematical tools you will need to solve the review session problem, without solving the problem itself.

1. Gaussian Vector Properties The problem involves inputs sampled from a unit Gaussian. Let $x \in \mathbb{R}^d$ where $x \sim \mathcal{N}(0, I_d)$.

- What is the expected squared ℓ_2 norm, $\mathbb{E}[||x||_2^2]$?
- *Hint: Recall that the sum of d squared standard normal variables follows a Chi-squared distribution.*

> **Workspace:**

2. Rank of Gradient Updates The problem defines the raw gradient $\nabla_W f(W)$ as an outer product $g_i x_i^T$.

- What is the rank of the matrix $G = uv^T$ where $u \in \mathbb{R}^{d_{out}}$ and $v \in \mathbb{R}^{d_{in}}$?
- What are the non-zero singular values of this rank-1 matrix?

> **Workspace:**

3. The Sign Function & Matrices In SignSGD (and the problem provided), we take the sign of the update.

- If M is a rank-1 matrix (e.g., uv^T), is $\text{sign}(M)$ (applied element-wise) necessarily rank-1?
- *Check Lecture 6, Page 9 for a specific property of sign(outer product).*

> **Workspace:**

4. Scaling Law Intuition In Xavier/He initialization, we scale weights by $\frac{1}{\sqrt{d_{in}}}$ (or similar).

- If $y = Wx$ and entries in $W \sim \mathcal{N}(0, 1)$ and $x \sim \mathcal{N}(0, 1)$, does the variance of y grow, shrink, or stay constant as dimension d increases?
- *Concept Check:* Why do we divide by $\sqrt{d}$ (or variance $1/d$)?

> **Workspace:**

Part 3: Conceptual Check

Briefly answer to ensure you understand the "Why".

1. Feature Learning vs. Lazy Training Your review problem asks about "updates during training." In the "Lazy Training" regime (often associated with NTK), do the features (activations) change significantly? How does μP aim to differ from this?

> **Answer:**

2. Why not just use Adam? Lecture 6 compares optimizers on a "NanoGPT speedrun". Why might we want an optimizer derived from RMS norm (like Muon or specific μP scaling) instead of standard Adam?

> **Answer:**

Answer Key (Fold over before starting!)

Part 2:

1. **Expectation:** $\mathbb{E}[\|x\|_2^2] = d$. (Each element x_i^2 has expectation 1).
2. **Rank:** Rank is 1. The singular value is $\sigma_1 = \|u\|_2\|v\|_2$.
3. **Sign Rank:** Yes, $\text{sign}(uv^T) = \text{sign}(u)\text{sign}(v)^T$, which is also rank-1.
4. **Scaling:** The variance grows linearly with d. We divide by $\sqrt{d}$ (std dev) or $1/d$ (variance) to keep the output variance constant ($O(1)$) as d increases.

Part 3:

1. **Lazy Training:** Features barely change; the model behaves like a linear model around initialization. μP aims to ensure feature updates (Δh) are $O(1)$ (significant) so feature learning actually occurs.
2. **Why not Adam:** Standard Adam doesn't always scale perfectly across widths (optimum learning rate shifts). μP/Muon aims to stabilize the optimal hyperparameters so they transfer across model sizes.